

1 Definición técnica del stack

1.1 Aplicación de escritorio multiusuario

Versión del documento: 1.0
Fecha de referencia: 24 de agosto de 2026
Estado: Baseline aprobado

1.2 Arquitectura del sistema

La solución está compuesta por clientes de escritorio Wails, una API Go centralizada y una instancia MySQL centralizada. Cada PC se comunica directamente con la API mediante HTTPS/TLS y JSON. Las PCs no se comunican entre sí ni acceden directamente a MySQL.



Figura 1: Arquitectura multiusuario: conexiones paralelas desde cada PC hacia la API Go centralizada.

Capa	Componente	Ubicación
Cliente	Aplicación Wails	Cada PC
Interfaz	Svelte + TypeScript + Tailwind CSS	Cada PC
Servidor	API centralizada en Go	Servidor de aplicación
Persistencia	MySQL centralizado	Servidor de datos
Comunicación	HTTPS/TLS + JSON	Entre cliente y API

1.3 Stack aprobado

1.3.1 Aplicación de escritorio y frontend

Tecnología	Versión	Función
Go	1.27.0	Código de integración y lógica local
Wails	2.13.0	Aplicación de escritorio y puente entre Go y frontend
Svelte	5.56.10	Componentes y estado de la interfaz
TypeScript	7.0.2	Lenguaje del frontend
HTML	HTML5 / Living Standard	Estructura de la interfaz
CSS	CSS moderno	Presentación visual
Tailwind CSS	4.3.3	Utilidades y sistema de estilos
Vite	8.2.2	Desarrollo, transformación y build frontend
Plugin Svelte para Vite	7.3.0	Integración de Svelte con Vite
Plugin Vite para Tailwind	4.3.3	Integración de Tailwind con Vite

1.3.2 API y backend

Tecnología	Versión	Función
Go	1.27.0	Implementación de la API centralizada
net/http	Incluida en Go	Servidor HTTP
encoding/json	Incluida en Go	Serialización y lectura de JSON
context	Incluida en Go	Cancelación y límites de operaciones
log/slog	Incluida en Go	Logging estructurado
HTTP/HTTPS	Estándar HTTP/TLS	Comunicación cliente-servidor
JSON	Estándar JSON	Formato de solicitudes y respuestas

1.3.3 Datos y persistencia

Tecnología	Versión	Función
MySQL	9.7.3	Base de datos centralizada

Tecnología	Versión	Función
database/sql	Incluida en Go 1.27	Interfaz estándar para SQL
go-sql-driver/mysql	v1.10.0	Driver de comunicación con MySQL
sqlc	v1.31.1	Generación de código Go tipado desde SQL
golang-migrate	v4.19.1	Migraciones versionadas del esquema
SQL	SQL compatible con MySQL	Consultas y definición de datos

1.3.4 Entorno de desarrollo

Tecnología	Versión	Función
Node.js	24.19.0 LTS	Runtime de herramientas frontend
pnpm	11.23.0	Gestión de paquetes y scripts frontend
Docker Engine	29.7.2	Ejecución de contenedores
Docker Desktop	4.86.0, si corresponde	Entorno Docker para Windows/macOS
Docker Compose	v5.5.0	Definición de servicios locales

Docker Compose se utilizará para ejecutar MySQL en el entorno local de desarrollo. MySQL de producción será centralizado en el servidor de datos o en un servicio administrado.

1.3.5 Calidad y pruebas

Tecnología	Versión	Función
ESLint	10.9.0	Análisis estático del frontend
typescript-eslint	8.67.0	Reglas de TypeScript para ESLint
eslint-plugin-svelte	3.23.0	Reglas de ESLint para Svelte
Vitest	4.1.11	Pruebas unitarias y de componentes frontend
gofmt	Incluido en Go 1.27	Formato del código Go
go vet	Incluido en Go 1.27	Análisis básico del código Go
go test	Incluido en Go 1.27	Pruebas del backend Go

1.3.6 Versionado, colaboración y automatización

Tecnología	Versión	Función
Git	2.55.0	Control de versiones local
GitHub	Servicio gestionado	Repositorio remoto y colaboración
GitHub Actions	Versiones por workflow	Integración continua, builds y releases
Markdown	Variante documentada	Documentación técnica

1.4 Límites de responsabilidad

Componente	Responsabilidad	No realiza
Svelte	Renderizar pantallas y gestionar estado visual	Acceder directamente a MySQL
Wails	Empaquetar el cliente y conectar frontend con Go local	Servir la API centralizada
Go cliente	Ejecutar integración local y consumir la API	Aplicar seguridad exclusivamente en el cliente
API Go	Autenticación, autorización, validación, negocio y acceso a datos	Exponer credenciales de MySQL
database/sql + driver	Abrir conexiones, consultas y transacciones desde Go	Definir la lógica visual
sqlc	Generar código Go desde SQL	Sustituir MySQL o las migraciones
golang-migrate	Aplicar cambios versionados al esquema	Crear copias de seguridad
MySQL	Persistir datos y coordinar transacciones	Servir directamente a las PCs
Docker Compose	Reproducir servicios del entorno	Sustituir el entorno de producción
GitHub Actions	Automatizar validaciones y builds	Sustituir la revisión técnica

1.5 Decisiones de integración

Decisión	Definición
Comunicación	Cliente Wails → HTTPS/TLS → API Go
Formato	JSON
Acceso a datos	API Go → red privada → MySQL
Cliente de base de datos	Prohibido en las aplicaciones Wails
Dependencias frontend	Administradas exclusivamente con pnpm
Lockfile frontend	pnpm-lock.yaml
Dependencias Go	go.mod y go.sum
Cambios de esquema	Migraciones SQL con golang-migrate
Pruebas backend	go test, con gofmt y go vet
Pruebas frontend	Vitest, TypeScript y ESLint
Entorno local	Docker Engine + Docker Compose + MySQL
Producción	API Go y MySQL centralizados

1.6 Elementos fuera del baseline

No forman parte del stack inicial: React, Vue, Angular, Tauri, Rust, Electron, otro ORM, Goose, Redis, Chi, Gin, Echo, Playwright, Testing Library, Swagger, Postman, Insomnia, Caddy y Nginx.

Su incorporación requerirá una modificación explícita de esta especificación. Goose no se utilizará junto con golang-migrate, y GORM o Ent no se utilizarán junto con sqlc sin una decisión arquitectónica nueva.

1.7 Baseline definitivo

```
Go 1.27.0
Wails 2.13.0
Svelte 5.56.10
TypeScript 7.0.2
HTML5 / HTML Living Standard
CSS moderno
Tailwind CSS 4.3.3
Vite 8.2.2
Node.js 24.19.0 LTS
pnpm 11.23.0
MySQL 9.7.3
go-sql-driver/mysql v1.10.0
```

```
database/sql  
sqlc v1.31.1  
golang-migrate v4.19.1  
Docker Engine 29.7.2  
Docker Compose v5.5.0  
Git 2.55.0  
GitHub  
GitHub Actions  
ESLint 10.9.0  
Vitest 4.1.11  
gofmt · go vet · go test  
HTTPS/TLS · JSON · SQL · Markdown
```

1.8 Fuentes de versión

[Wails](#) · [Go](#) · [Vite](#) · [Tailwind CSS](#) · [Node.js](#) · [MySQL](#) · [Docker Engine](#) · [Docker Compose](#)